INTRODUCTION TO POLICY GRADIENT METHODS

INTERMEDIATE DEEP LEARNING

Zhuofan XU
AI / Reinforcement Learning Engineer @ Deepmath
PhD in AI - Université Paris-Saclay · ENS Paris-Saclay

PREVIOUSLY…

FROM Q-LEARNING TO DEEP Q-LEARNING

PREVIOUSLY …

FROM Q-LEARNING TO DEEP Q-LEARNING

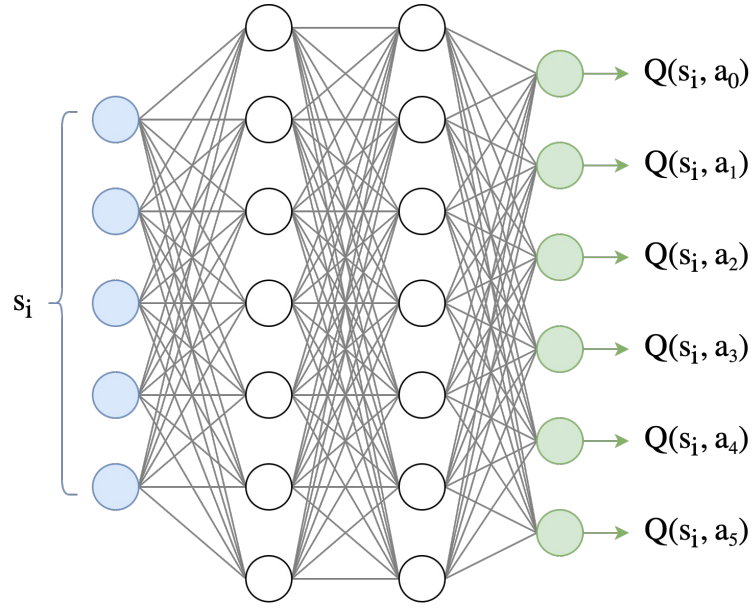
- During our last two sessions, we covered some fundamental concepts of:
  - Reinforcement Learning (Markov decision process, interactive learning paradigm)
  - Tabular Q-learning (Update Q-value through Temporal Difference)

$$Q(s,a) \leftarrow Q(s,a) + \alpha \underbrace{[\, r + \gamma \max_{a'} Q\,(s', a') - Q(s,a)\,]}_{\text{TD Error } \delta}$$

PREVIOUSLY …

FROM Q-LEARNING TO DEEP Q-LEARNING

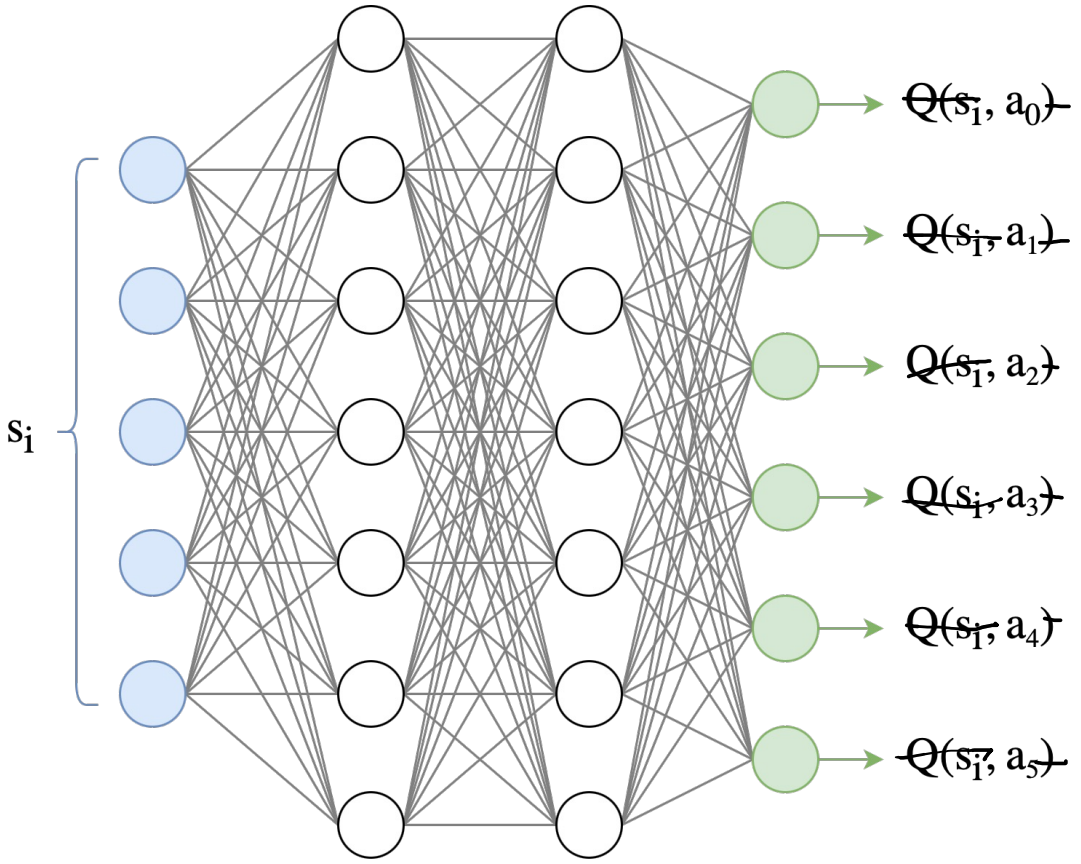
- During our last two sessions, we covered some fundamental concepts of:
- Deep Q Network (DQN)
  - Target Q-network, an offline network used to stabilize learning by fixing the target
  - Experience replay, improves data efficiency, reduces forgetting, and breaks temporal correlations between samples
- DQN Variants
  - Double DQN
  - Prioritized Experience Replay



PREVIOUSLY …

FROM Q-LEARNING TO DEEP Q-LEARNING

- What if the network predicts actions?



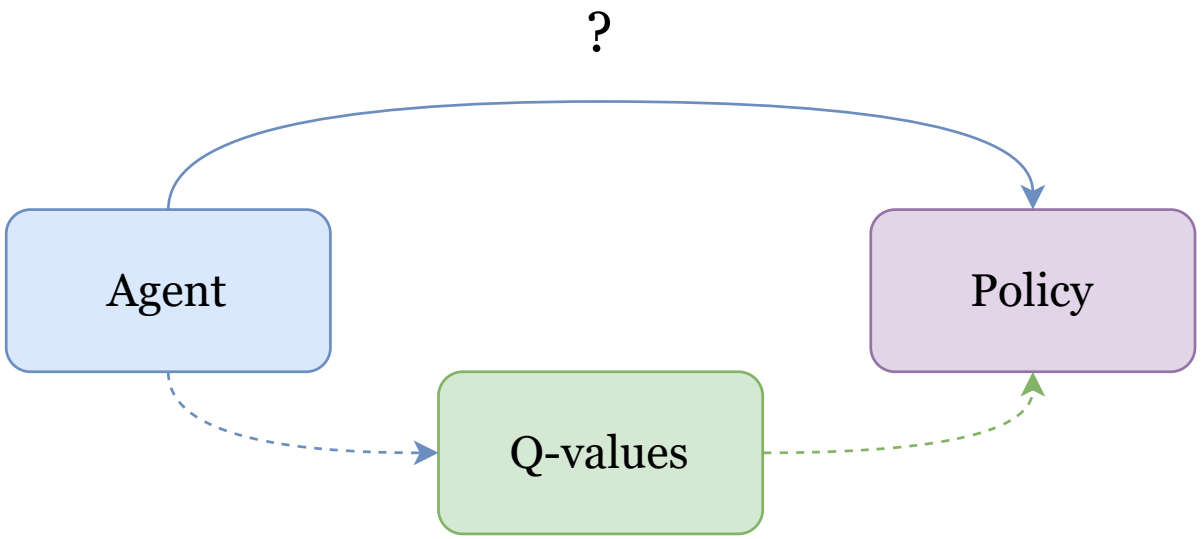
INTRODUCTION

FROM Q-LEARNING TO POLICY-BASED METHODS

INTRODUCTION

FROM Q-LEARNING TO POLICY-BASED METHODS

- In Q-learning, we first learn a Q-value function and then build the policy from it
- But what if we could skip the Q-values and learn the policy directly?
- Why bother?
  - Handles continuous action spaces
  - Supports stochastic policies
  - …



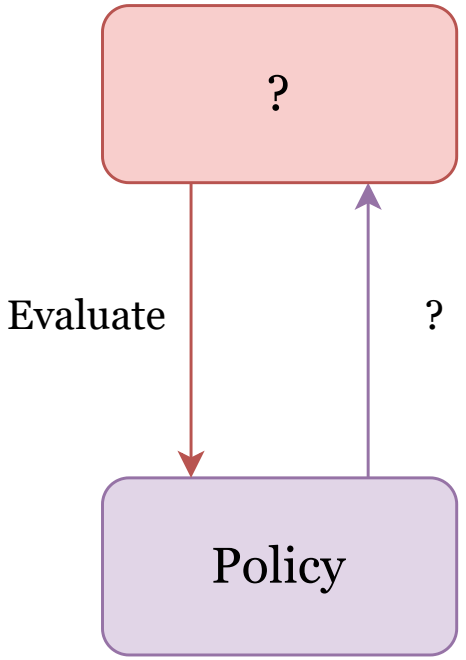
INTRODUCTION

FROM Q-LEARNING TO POLICY-BASED METHODS

- How do we calculate the gradient of a policy?
- Or in other words, how do we judge if a policy is good or bad?
- Given a trajectory

$$\tau = (s_0, a_0, r_0, \cdots, s_T, a_T, r_T)$$

- …



THE OBJECTIVE FUNCTION

- Given a trajectory

$$\tau = (s_0, a_0, r_0, \cdots, s_T, a_T, r_T)$$

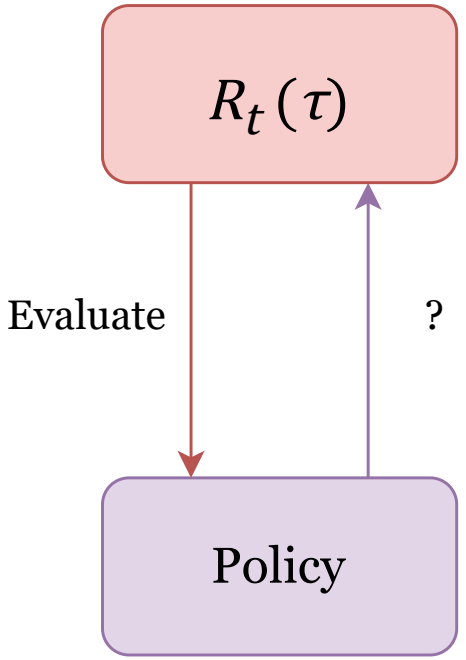
- We calculate the return-to-go from a trajectory

$$R_t(\tau) = \sum_{k=t}^{T} \gamma^{k-t} r_k$$

- And $R(\tau) = R_0(\tau)$
- Same idea with the return we see in the session 1:

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$$

- But $R_t(\tau)$ depends on trajectory τ and current time step t



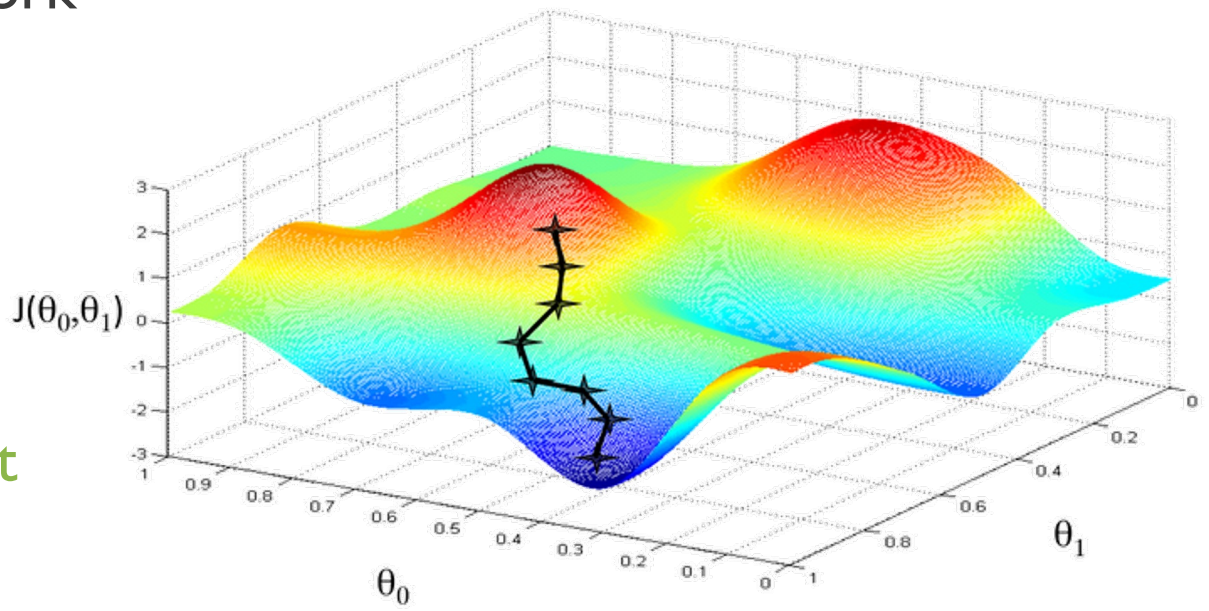
INTRODUCTION

THE POLICY GRADIENT

- A policy is approximated by a neural network consisting of learnable parameters θ: π_θ
- Our goal is to find the set of parameter θ that maximize the expected $R(\tau)$:

$$J(\pi_\theta) = E_{\tau \sim \pi_\theta}[R(\tau)]$$

- To achieve this, we perform gradient ascent on $J(\pi_\theta)$
- *Recall:* Gradient points in the direction of steepest ascent



INTRODUCTION

THE POLICY GRADIENT

- The term $\nabla_\theta J(\pi_\theta)$ is known as the policy gradient, and is defined as:

$$\nabla_\theta J(\pi_\theta) = E_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} R_t(\tau) \, \nabla_\theta \log \pi_\theta \left(a_t | s_t \right) \right]$$

- Intuition:
  - If $R_t(\tau)$ is positive …
  - If $R_t(\tau)$ is negative …
- Where does the log term come from?

INTRODUCTION
LOG-DERIVATIVE TRICK 😎

- Starting point: $\nabla_\theta J(\pi_\theta) = \nabla_\theta E_{\tau\sim\pi_\theta}[R(\tau)]$

$$= \nabla_\theta \sum_\tau p_\theta(\tau)\cdot R(\tau)$$

  - Definition of expectation

$$= \sum_\tau R(\tau)\cdot \nabla_\theta\, p_\theta(\tau)$$

  - $R(\tau)$ does not explicitly depend on θ when τ is fixed

$$= \sum_\tau R(\tau)\; p_\theta\,(\tau)\cdot \frac{\nabla_\theta\, p_\theta(\tau)}{p_\theta(\tau)}$$

  - Log-derivative trick

$$= \sum_\tau R(\tau)\cdot p_\theta\,(\tau)\cdot \nabla_\theta \log p_\theta\,(\tau)$$

  - Definition of expectation

$$= E_{\tau\sim\pi_\theta}[R(\tau)\cdot \nabla_\theta \log p_\theta\,(\tau)]$$

INTRODUCTION

LOG-DERIVATIVE TRICK 😎

$$\log p_\theta(\tau) = \log \prod_{t=0}^{T} \underbrace{P(s_{t+1} \mid s_t, a_t)}_{\text{Transition in env}} \cdot \pi_\theta(a_t \mid s_t)$$

$$= \sum_{t=0}^{T} \left(\log P(s_{t+1} \mid s_t, a_t) + \log \pi_\theta(a_t \mid s_t)\right)$$

From the slide before

$$\underbrace{E_{\tau\sim\pi_\theta}[R(\tau) \cdot \nabla_\theta \log p_\theta(\tau)]}_{\text{From the slide before}} = E_{\tau\sim\pi_\theta}\left[R(\tau) \cdot \nabla_\theta \sum_{t=0}^{T} \underbrace{\left(\log P(s_{t+1} \mid s_t, a_t) + \log \pi_\theta(a_t \mid s_t)\right)}_{\text{Does not depend on } \theta}\right]$$

$$= E_{\tau\sim\pi_\theta}\left[R(\tau) \cdot \nabla_\theta \sum_{t=0}^{T} \log \pi_\theta(a_t \mid s_t)\right]$$

$$= E_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \underbrace{R_t(\tau)}_{a_t \text{ does not influence reward before } t} \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

REINFORCE

MONTE CARLO SAMPLING – REINFORCE ALGORITHM

REINFORCE

MONTE CARLO SAMPLING

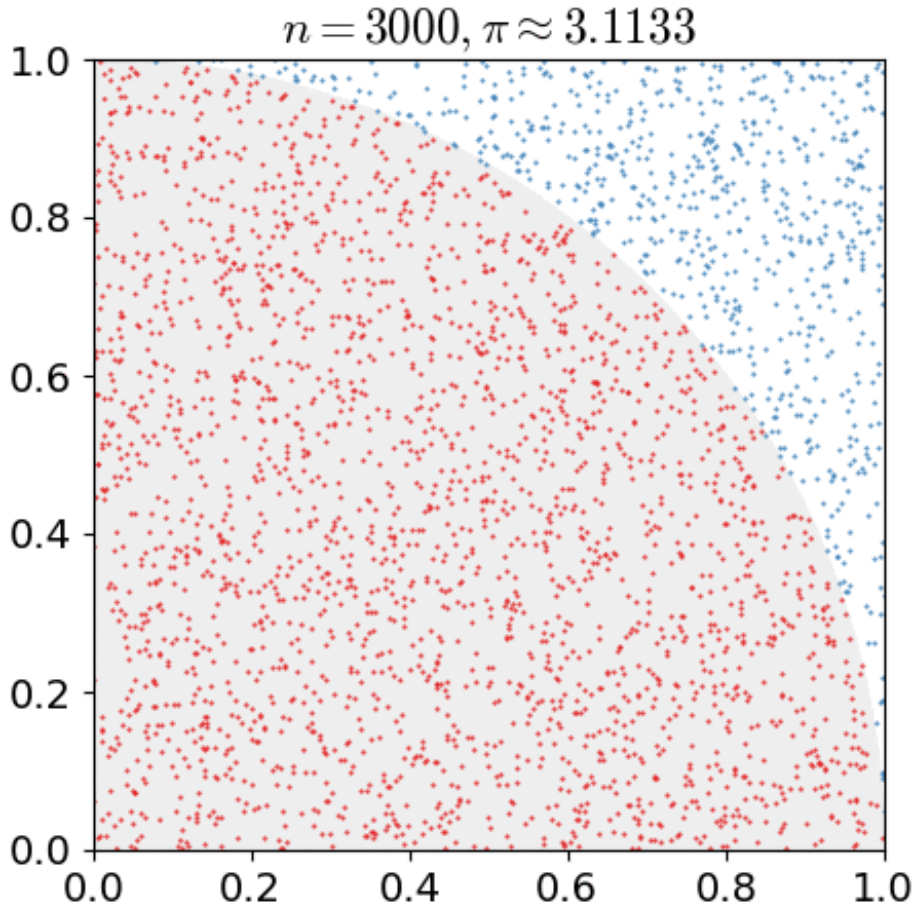
- Proposed in the 1940s by a group of scientists working on the Manhattan Project
- Approximation with random Sampling
- Named after the Monte Carlo Casino (one scientist's uncle loved gambling there 🎲😅)



REINFORCE

MONTE CARLO SAMPLING

- $\frac{\text{Sector Area}}{\text{Square Area}} = \frac{\pi}{4}$



REINFORCE ALGORITHM

- REINFORCE is a Monte Carlo-based policy gradient algorithm
- It directly optimizes the expected return by adjusting the policy parameters in the direction of the gradient
- The algorithm increases the probability of actions that yield higher returns and decreases it for those with lower returns

REINFORCE ALGORITHM

Algorithm 1: REINFORCE Algorithm (Monte Carlo Policy Gradient)

Input: Learning rate α, discount factor γ, number of episodes N
Output: Trained policy parameters θ

Initialize policy parameters θ randomly;
for $n = 1, 2, \ldots, N$ **do**
  Generate one full episode $\tau = (s_0, a_0, r_0, \ldots, s_T, r_T, r_T)$ by following π_θ;
  for $t = 0$ **to** $T - 1$ **do**
    Compute return-to-go:;

$$R_t(\tau) \leftarrow \sum_{k=t}^{T} \gamma^{k-t} r_k$$

  Update policy parameters:;

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T} R_t(\tau) \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

REINFORCE ALGORITHM: REMARKS

- "REINFORCE" stands for "REward Increment = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility" 😅
- No explicit exploration mechanism (e.g. ϵ-greedy)
- No value function
- Can update after one episode or several episodes
- On-policy algorithm
  - learns from data collected by its current policy
  - → experience replay not possible
- High variance in gradient estimates

CODING

TD4: REINFORCE

TO BE CONTINUED …

DRAWBACKS OF REINFORCE

TO BE CONTINUED …

DRAWBACKS OF REINFORCE

- High variance
- Credit assignment problem
- Inefficient updates (only after an episode finishes)

